

Rapport Technique — BankApp Spring Boot + React

Module : Spring

Projet : Application bancaire avec backend Spring Boot REST API et frontend React

Étudiant : El-Mehdi Taoufik

Dépôt GitHub : <https://github.com/Hasanpiker5543/spring-bank-project>

1. Introduction

Ce rapport présente la réimplémentation d'une application de gestion bancaire en utilisant l'écosystème Java/Spring pour le backend et React pour le frontend.

Le projet respecte les consignes du module Spring : backend Spring Boot exposant des services REST API, frontend moderne consommant ces API, dépôt GitHub, README, rapport technique et vidéo de démonstration.

2. Objectif du projet

L'objectif est de proposer une application bancaire permettant la gestion des clients, des comptes, des transactions et des transferts d'argent. L'application met également en place une authentification simple avec séparation des rôles administrateur et client.

3. Fonctionnalités principales

Les fonctionnalités couvrent les besoins essentiels d'une application bancaire pédagogique.

- Authentification : login, register, logout
- Dashboard administrateur
- Dashboard client
- Gestion des clients
- Gestion des comptes bancaires
- Consultation des transactions
- Transfert d'argent
- Filtrage des données selon le rôle
- Interface React moderne

4. Architecture générale

Le projet suit une architecture séparée backend/frontend. Le frontend React communique avec le backend Spring Boot via des requêtes HTTP vers des endpoints REST. Le backend gère la logique métier et la persistance avec Spring Data JPA et H2.

Utilisateur -> Frontend React/Vite -> REST API Spring Boot -> Services -> Repositories -> H2 Database

5. Architecture backend Spring Boot

Le backend est organisé en couches : controllers pour exposer les API REST, services pour la logique métier, repositories pour l'accès aux données, models pour les entités JPA et DTO pour les échanges de données.

```
backend/src/main/java/com/bankapp/  
■■■■ controller/  
■■■■ service/  
■■■■ repository/  
■■■■ model/  
■■■■ dto/  
■■■■ config/
```

6. Architecture frontend React

Le frontend React propose une interface moderne avec page de connexion, inscription, tableau de bord, cartes de statistiques, liste des comptes, transactions récentes et formulaire de transfert. Axios est utilisé pour consommer les API REST.

```
frontend/src/  
■■■ App.jsx  
■■■ main.jsx  
■■■ styles.css  
■■■ services/api.js
```

7. Choix techniques

Spring Boot a été choisi pour sa robustesse et sa capacité à créer rapidement des API REST. React et Vite ont été choisis pour créer une interface rapide, moderne et maintenable. H2 permet une base de données simple pour la démonstration.

8. Modèle de données

Le modèle de données repose sur quatre entités principales : Client, Account, Transaction et AppUser.

```
Client(id, username, phone)  
Account(id, accountNumber, balance, client_id)  
Transaction(id, type, amount, date, account_id)  
AppUser(id, username, password, role, client_id)
```

9. API REST

Le backend expose les ressources clients, comptes, transactions, transferts et authentification via des endpoints REST.

```
POST /api/auth/login  
POST /api/auth/register  
GET /api/clients  
POST /api/clients  
GET /api/accounts  
POST /api/accounts  
GET /api/transactions  
POST /api/transfers
```

10. Authentification et rôles

L'application distingue deux rôles : ADMIN et CLIENT. L'administrateur peut voir tous les clients et comptes, ajouter des clients et créer des comptes. Le client ne voit que ses propres informations bancaires.

11. Transfert d'argent

Le transfert débite le compte source et crédite le compte destinataire. Le backend vérifie l'existence des comptes, le montant et le solde disponible avant d'effectuer l'opération.

12. Base de données H2

La base H2 en mémoire est initialisée avec des données de démonstration dans data.sql. Elle permet de tester rapidement l'application sans installation de serveur de base de données externe.

13. Lancement du projet

Le backend se lance avec Maven Wrapper et le frontend avec npm. Le projet contient aussi une configuration Docker Compose.

```
Backend:  
cd backend  
.\mvnw.cmd spring-boot:run  
  
Frontend:  
cd frontend  
npm.cmd install  
npm.cmd run dev
```

14. Tests et validation

Un test Spring Boot vérifie le chargement du contexte backend. Le frontend est validé avec une construction de production Vite.

15. Conclusion

Ce projet répond aux consignes du module Spring : il fournit un backend Spring Boot REST API, un frontend React moderne, une base de données, une documentation README, un rapport technique et une vidéo de démonstration.

16. Annexes

Lien GitHub : <https://github.com/Hasanpiker5543/spring-bank-project>

Backend : <http://localhost:8080>

Frontend : <http://localhost:5173>

Console H2 : <http://localhost:8080/h2-console>